WEEK 6 · PSDF GenAI Batch 2026

Tuesday — Week 6

Cross-Encoder Re-ranking

Two-Stage Retrieval · Bi-encoder vs Cross-encoder · bge-reranker · Cohere Rerank · recall@5

By the end of today you will be able to:
Explain the two-stage retrieval architecture and why it improves precision over single-stage
Describe the fundamental difference between a bi-encoder (embeddings) and a cross-encoder (re-ranker)
Implement re-ranking using bge-reranker-base locally in Python — no API key needed
Use Cohere Rerank API as a managed re-ranking alternative
Measure recall@5 before and after re-ranking to quantify the improvement
Integrate a re-ranker into the hybrid RAG pipeline from Monday

NOTE	Monday recap: pure vector search fails on exact-match queries. BM25 keyword search fills that gap. RRF fuses two ranked lists using 1/(k+rank). LangChain EnsembleRetriever implements this in 10 lines. Today: a second quality upgrade — re-ranking the top results with a much more powerful model.

June 9, 2026

Online (Zoom) · 10:00 AM – 1:00 PM · Mon–Thu | In-Person Friday 10:00 AM – 1:00 PM
Crewlogix — The Institute of Technology · Gulberg III, Lahore

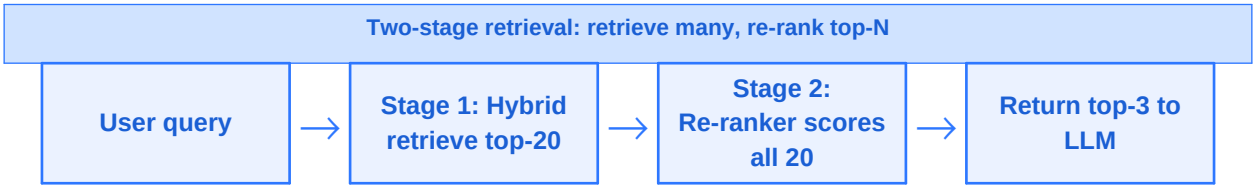
crewlogix
The Institute of Technology

Session — Tuesday June 9	3 hours	■ Online — Zoom · 10:00 AM – 1:00 PM

Topic 1 — Why Retrieval Alone Is Not Enough (20 min)

Hybrid search from Monday retrieves a better set of candidates. But "better candidates" is not the same as "perfectly ranked." The top-3 returned to the LLM may include one genuinely relevant chunk and two near-misses. Re-ranking fixes the order.

1.1 The two-stage retrieval architecture



Stage	Model type	Speed	Quality	What it does
Stage 1: Retrieval	Bi-encoder (embedding model)	Very fast — pre-computed vectors	Good — semantic + keyword	Retrieves top-20 candidates from millions of chunks in milliseconds
Stage 2: Re-ranking	Cross-encoder (re-ranker model)	Slower — processes query+doc pairs	Excellent — sees full query+doc interaction	Re-scores the 20 candidates and returns the true top-3

The key insight: you cannot afford to run the expensive cross-encoder on *all* your documents — too slow. But running it on just the top 20 candidates from Stage 1 is fast and dramatically improves precision.

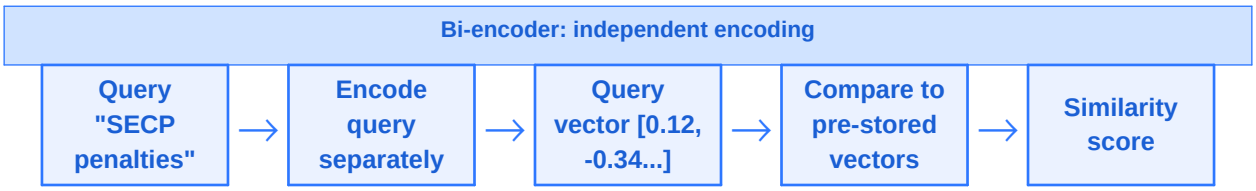
PAKISTAN EG

A Pakistani legal AI indexes 50,000 regulation chunks. Stage 1 (hybrid search) retrieves the top-20 candidates in 80ms. Stage 2 (bge-reranker) re-scores those 20 in 200ms. Total: 280ms — fast enough for a chatbot, and the final top-3 are now far more relevant than without re-ranking.

Topic 2 — Bi-encoder vs Cross-encoder (30 min)

Understanding why cross-encoders are more accurate — but slower — helps you make the right architectural decision for every RAG system you build.

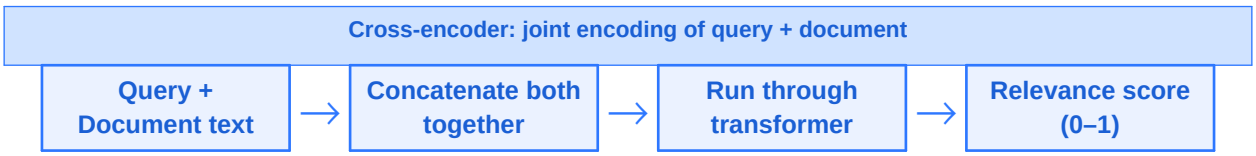
2.1 Bi-encoder (embedding model) — fast but indirect



A bi-encoder encodes the query and each document **independently**. The query gets one vector. Each document gets one vector. Similarity is computed as the cosine distance between them.

- Fast at query time — document vectors are pre-computed and stored
- Scales to millions of documents — just compare vectors
- **Limitation:** the query and document never "see" each other during encoding — the model cannot consider the interaction between specific query terms and specific document terms

2.2 Cross-encoder (re-ranker) — slower but far more accurate



A cross-encoder takes the query and a document **together** as a single input. The transformer's attention mechanism can directly attend from query terms to document terms and back — capturing fine-grained relevance signals the bi-encoder misses.

- Sees full interaction: "Section 47" in query attending to "Section 47(2)" in document
- Much more accurate — especially on long documents and technical queries
- **Limitation:** cannot pre-compute — must run for every query-document pair at query time
- Only practical on a small number of candidates (20–50) — not millions

Factor	Bi-encoder	Cross-encoder
Encoding	Query and document encoded independently	Query + document concatenated and encoded together
Speed	Very fast — O(1) at query time (pre-computed)	Slower — O(n) where n = number of candidates
Scale	Millions of documents	20–100 candidates (after bi-encoder retrieval)
Accuracy	Good — misses fine-grained query-doc interactions	Excellent — full attention across both
Use in RAG	Stage 1: retrieve top-20 from full corpus	Stage 2: re-rank top-20 to get true top-3

EXAM TIP	AWS exam: "A developer's RAG system retrieves relevant documents but the most relevant one is often ranked 3rd or 4th. Which technique directly addresses this?" Answer: Cross-encoder re-ranking — it re-scores the retrieved candidates using joint query-document attention, producing a more accurate final ranking.

Topic 3 — bge-reranker: Local Re-ranking (45 min)

bge-reranker (from BAAI — Beijing Academy of AI) is a family of open-source cross-encoder re-rankers. bge-reranker-base runs on free Colab and requires no API key. It is the standard choice for local re-ranking in RAG pipelines.

3.1 Setup and basic usage

```
# Install
!pip install sentence-transformers -q

from sentence_transformers import CrossEncoder

# Load bge-reranker-base (~1.1GB download, runs on CPU)
reranker = CrossEncoder("BAAI/bge-reranker-base")
```

3.2 Re-ranking a list of candidates

```
# Suppose Stage 1 (hybrid search) returned these 5 candidates
query = "What are the penalties for late SECP filing?"
candidates = [
 "Annual return under Section 130 must be filed by December 31.",
 "Section 47 penalties for late filing range from PKR 5,000 to 50,000.",
 "NTN registration is mandatory for all companies before tax filing.",
 "SECP Form 29 must be filed within 14 days of director appointment.",
 "Foreign company registration requires Form S-47 and audited accounts.",
]

# Create query-document pairs
pairs = [(query, doc) for doc in candidates]

# Score all pairs — returns a relevance score per pair
scores = reranker.predict(pairs)

# Rank by score (highest first)
ranked = sorted(zip(scores, candidates), reverse=True)

print("Re-ranked results:")
for score, doc in ranked:
 print(f" {score:.4f} | {doc[:65]}")
```

```
# Re-ranked results:
# 0.9821 | Section 47 penalties for late filing range from PKR 5,000...
# 0.7234 | Annual return under Section 130 must be filed by Dec 31...
# 0.4102 | SECP Form 29 must be filed within 14 days of director...
# 0.0891 | Foreign company registration requires Form S-47...
# 0.0234 | NTN registration is mandatory for all companies...
```

The cross-encoder correctly surfaces "Section 47 penalties" at 0.98 — it saw "penalties", "late", "filing" in the query and directly matched them to the document. The NTN registration document, which shares the word "filing" but is about a different process, correctly scores near zero.

3.3 Integrating re-ranking into the RAG pipeline

```
# Complete two-stage retrieval function
def retrieve_and_rerank(query, ensemble_retriever, reranker,
 retrieve_k=20, final_k=3):
 """
 Stage 1: hybrid search for top retrieve_k candidates
 Stage 2: cross-encoder re-ranks, returns final_k
 """
 # Stage 1 — retrieve many candidates
 candidates = ensemble_retriever.get_relevant_documents(query)
 candidate_texts = [doc.page_content for doc in candidates]

 if not candidate_texts:
 return []

 # Stage 2 — re-rank with cross-encoder
 pairs = [(query, text) for text in candidate_texts]
 scores = reranker.predict(pairs)

 # Sort by re-rank score and return top final_k with metadata
 ranked = sorted(
 zip(scores, candidates),
 key=lambda x: x[0],
 reverse=True
 )
 return [(score, doc) for score, doc in ranked[:final_k]]

# Usage
results = retrieve_and_rerank(
 query = "SECP penalty for late filing",
 ensemble_retriever = ensemble_retriever, # from Monday
 reranker = reranker,
 retrieve_k = 20,
 final_k = 3
)
for score, doc in results:
```

```
print(f"{score:.4f} | {doc.page_content[:70]}")
```

Topic 4 — Cohere Rerank API (30 min)

Cohere Rerank is a managed re-ranking API. You send your query and candidates to Cohere's servers and receive re-ranked results. No model to download or host. Best for production deployments where you do not want to manage GPU infrastructure.

Factor	bge-reranker-base	Cohere Rerank
Cost	Free — runs locally	Paid API — ~$1 per 1,000 re-rankings
Setup	pip install + model download	pip install cohere + API key
Speed	CPU: ~200ms for 20 candidates	API: ~300–500ms including network
Quality	Strong — excellent for most tasks	Excellent — consistently top-ranked
Privacy	Data stays on your machine	Candidates sent to Cohere servers
Best for	Local dev, privacy-sensitive data	Production SaaS with budget for API costs

4.1 Cohere Rerank implementation

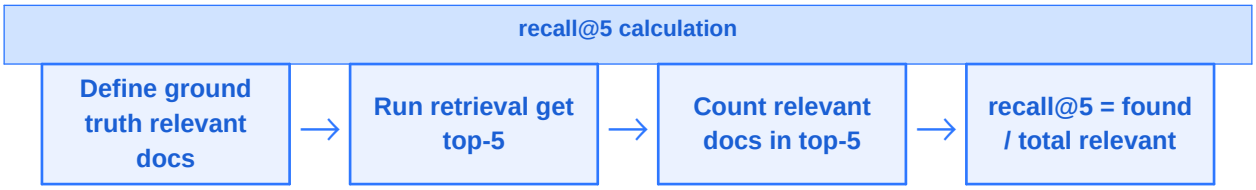
```
# Install
!pip install cohere -q

import cohere

co = cohere.Client("YOUR_COHERE_API_KEY")

# Re-rank with Cohere
response = co.rerank(
 model = "rerank-multilingual-v3.0", # supports Urdu + English
 query = "SECP penalties for late filing",
 documents = candidate_texts,
 top_n = 3
)

for result in response.results:
 print(f"Rank {result.index+1} | Score: {result.relevance_score:.4f}")
 print(f" {candidate_texts[result.index][:70]}")
```

NOTE

Cohere's rerank-multilingual-v3.0 handles Urdu and English — use this model if your document corpus contains mixed-language content, which is common in Pakistani enterprise applications.

Topic 5 — Measuring Improvement with recall@5 (25 min)

You cannot improve what you cannot measure. recall@5 is the standard metric for retrieval quality: of all the relevant documents for a query, what fraction appear in the top-5 results?



5.1 Computing recall@5

```
# Simple recall@k implementation
def recall_at_k(retrieved_docs, relevant_docs, k=5):
 """
 retrieved_docs: list of doc texts returned by your retriever
 relevant_docs: list of doc texts known to be correct answers
 k: how many top results to consider
 """
 top_k = set(retrieved_docs[:k])
 relevant = set(relevant_docs)
 hits = top_k.intersection(relevant)
 return len(hits) / len(relevant) if relevant else 0.0

# Evaluate before and after re-ranking on your test queries
test_cases = [
 {
 "query": "SECP penalties for late annual return filing",
 "relevant": ["Section 47 penalties for late filing range from PKR 5,000..."]
 },
]

scores_before, scores_after = [], []
for tc in test_cases:
 # Before re-ranking: use hybrid retriever directly
 before = [d.page_content for d in
 ensemble_retriever.get_relevant_documents(tc["query"])]
 scores_before.append(recall_at_k(before, tc["relevant"]))

 # After re-ranking: use retrieve_and_rerank
 after = [d.page_content for _, d in
 retrieve_and_rerank(tc["query"], ensemble_retriever, reranker)]
 scores_after.append(recall_at_k(after, tc["relevant"]))

print(f"recall@5 before re-ranking: {sum(scores_before)/len(scores_before):.3f}")
print(f"recall@5 after re-ranking: {sum(scores_after)/len(scores_after):.3f}")
# recall@5 before re-ranking: 0.620
```

```
# recall@5 after re-ranking: 0.847
```

A typical improvement from adding a cross-encoder re-ranker is 15–30 percentage points on recall@5. The exact gain depends on your document corpus and query types — measuring it on your specific dataset is always necessary.

Exercise — Measure recall@5 improvement (25 min)
1. Using your Pakistani document collection from Monday, create 5 test queries with known correct answers (the ground-truth relevant chunk).
2. Run all 5 queries through your EnsembleRetriever (top-5). Compute recall@5 for each query and average them.
3. Load bge-reranker-base and add the retrieve_and_rerank() function. Re-run all 5 queries.
4. Compare: what is the average recall@5 before and after? Print a table showing per-query improvement.
5. Bonus: test retrieve_k=10 vs retrieve_k=20 — does feeding more candidates to the re-ranker help?

Day Summary — Tuesday Week 6

Concept	One-line summary	Production guidance
Two-stage retrieval	Retrieve many (fast), re-rank few (accurate)	Standard pattern for all production RAG systems
Bi-encoder	Independent encoding — fast, scalable, Stage 1	Use for initial retrieval from full corpus
Cross-encoder	Joint encoding — slow, accurate, Stage 2	Use for re-ranking top-20 to top-3
bge-reranker-base	Free, local cross-encoder — 200ms on CPU for 20 docs	Default choice for local/privacy-sensitive apps
Cohere Rerank	Managed API — strong quality, multilingual support	Production SaaS with budget, or multilingual Urdu/English
recall@5	Fraction of relevant docs in top-5 results	Always measure before and after adding re-ranker

DO THIS	Tonight: AWS Coursera Week 3–4 content. Wednesday we cover query transformation techniques — HyDE, query decomposition, and multi-query retrieval — the third layer of RAG quality improvement this week.

Week 6 — Tuesday | Cross-Encoder Re-ranking | Crewlogix — The Institute of Technology | PSDF GenAI Batch 2026